



11-DARS · 14 TADAN

Autentifikatsiya va avtorizatsiya

Parollarni xeshlash (bcrypt), sessiya va JWT, httpOnly cookie, himoyalangan marshrutlar, rollar va ruxsatlar, access/refresh tokenlar hamda frontendda tokenlar bilan ishlash.

O'rta–Yuqori · ~130 daqiqa

Deyarli har bir ilovada "Kirish" tugmasi bor. Uning ortida esa veb-xavfsizlikning eng nozik qismi yotadi: parollarni qanday saqlash, foydalanuvchini keyingi so'rovlarda qanday "tanish", kimga nimaga ruxsat berish. Bu yerdagi xato butun ma'lumotlar bazasining o'g'irlanishiga olib kelishi mumkin. Shuning uchun har bir qarorning **nima uchun** ekanini tushunamiz.

Autentifikatsiya va avtorizatsiya

	Autentifikatsiya (AuthN)	Avtorizatsiya (AuthZ)
Savol	Sen kimsan?	Senga nima qilish mumkin?
Misol	Login va parolni tekshirish	Faqat admin foydalanuvchini o'chira oladi
Xato kodi	401 Unauthorized	403 Forbidden
Ketma-ketlik	Birinchi	Keyin

Parollarni saqlash: xeshlash

Qoida №1: parolni hech qachon ochiq holda saqlamang. Baza o'g'irlansa (bu katta kompaniyalarda ham bo'ladi), barcha parollar oshkor bo'ladi — odamlar esa bir xil parolni ko'p saytda ishlatadi.

Nima uchun shifrlash emas, xeshlash?

- **Shifrlash** — ikki tomonlama: kalit bilan qayta ochish mumkin. Kalit o'g'irlansa — hamma parollar ochiladi.
- **Xeshlash** — bir tomonlama funksiya: `hash("saʼlom123")` → `"$2b$12$Kix..."`, lekin xeshdan parolni qaytarib bo'lmaydi. Tekshirishda kiritilgan parol qayta xeshlanadi va solishtiriladi.

Nima uchun MD5/SHA-256 emas?

Ular **tez** ishlash uchun yaratilgan — zamonaviy GPU sekundiga milliardlab SHA-256 hisoblaydi va oddiy parollarni lug'at hujumi bilan bir zumda topadi. Parollar uchun ataylab **sekin** va **tuzli (salted)** algoritmlar kerak:

- **bcrypt** — eng ko'p ishlatiladigan; "cost factor" (masalan 12) bilan sekinlik sozlanadi.
- **argon2id** — zamonaviy tavsiya (OWASP), xotirani ham talab qiladi.
- **Tuz (salt)** — har bir parolga qo'shiladigan tasodifiy qiymat. Bir xil parollar har xil xeshga ega bo'ladi, tayyor "rainbow table"lar ishlamaydi. bcrypt tuzni avtomatik yaratadi va xesh ichida saqlaydi.

JavaScript

```
import bcrypt from 'bcrypt'; // npm install bcrypt (yoki sof JS: bcryptjs)

const hash = await bcrypt.hash('MeningParolim2026', 12);
// '$2b$12$C6UzMDM.H6dfI/f/IKcEe05H8v.Lk3PqxW7JLw7xYcR9EJzt4P6eW'
// |   |   | 22 belgili tuz + 31 belgili xesh
// |   |   | cost = 12 (2^12 = 4096 iteratsiya)
// |   |   | algoritm versiyasi

const ok = await bcrypt.compare('MeningParolim2026', hash); // true
const bad = await bcrypt.compare('boshqa', hash); // false
```

Foydalanuvchini "eslab qolish": sessiya va token

HTTP holatsiz (1-dars) — server har bir so'rovda kim kelganini bilmaydi. Login'dan keyin mijozga "guvohnoma" beriladi va u har bir so'rov bilan uni ko'rsatadi. Ikki asosiy yondashuv bor:

1. Sessiyalar (stateful)

- 1. Login muvaffaqiyatli → server tasodifiy **sessiya ID** yaratadi va uni **serverda** (Redis yoki bazada) saqlaydi: `sid_abc → { userId: 7 }`.
- 2. ID mijozga `Set-Cookie: sid=abc; HttpOnly; Secure` orqali yuboriladi.
- 3. Brauzer har so'rovda cookie'ni avtomatik yuboradi; server ID bo'yicha foydalanuvchini topadi.
- 4. Chiqish — serverdagi yozuvni o'chirish. Darhol, ishonchli.

2. JWT tokenlar (stateless)

JWT (JSON Web Token) — server **imzolagan** va ichida ma'lumot saqlovchi satr. Server hech narsani eslab qolmaydi: imzoni tekshirish kifoya.

Matn

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI3Iiwicm9sZSI6InVzZXIiLCJleHAiOjE3OTE5ODAwMDB9.Xk3v...
|----- header -----| |----- payload -----|
|-----| |----- imzo -----|

header: { "alg": "HS256", "typ": "JWT" }
payload: { "sub": "7", "role": "user", "iat": 1791979100, "exp": 1791980000 }
imzo:    HMAC-SHA256(base64(header) + "." + base64(payload), JWT_SECRET)
```

 **JWT shifrlanmagan!**

Header va payload shunchaki Base64 — jwt.io da istalgan kishi o'qiy oladi. Imzo faqat **o'zgartirilmaganini** kafolatlaydi. Payload'ga parol, karta raqami kabi maxfiy ma'lumot qo'ymang.

	Sessiya	JWT
Holat qayerda	Serverda	Tokenning o'zida
Har so'rovda bazaga murojaat	Ha (Redis bilan tez)	Yo'q
Chiqish/bloklash	Darhol	Token muddati tugaguncha amal qiladi (qisqa muddat + refresh token kerak)

Bir nechta server/mikroservis	Umumiy sessiya ombori kerak	Oson — har bir servis imzoni tekshiradi
Mobil ilovalar	Mumkin	Juda qulay

Token qayerda saqlanadi?

Joy	XSS'ga qarshi	CSRF'ga qarshi	Izoh
<code>localStorage</code>	❌ Har qanday skript o'qiydi	✅ Avtomatik yuborilmaydi	Oddiy, lekin XSS bo'lsa token o'g'irlanadi
<code>HttpOnly</code> cookie	✅ JS o'qiy olmaydi	⚠️ <code>SameSite</code> kerak	Veb-ilovalar uchun tavsiya
Xotira (JS o'zgaruvchisi)	⚠️ Qisman	✅	Sahifa yangilansa yo'qoladi — refresh token bilan birga

JavaScript

```
res.cookie('access_token', token, {
  httpOnly: true, // document.cookie orqali
  // o'qib bo'lmaydi
  secure: process.env.NODE_ENV === 'production', // faqat HTTPS orqali
  sameSite: 'lax', // boshqa saytdan POST
  // so'rovlarda yuborilmaydi (CSRF himoyasi)
  maxAge: 15 * 60 * 1000, // 15 daqiqa
  path: '/',
});
```

Access va refresh tokenlar

Muammo: uzoq muddatli token o'g'irlansa — hujumchi uzoq vaqt foydalanadi; qisqa muddatli bo'lsa — foydalanuvchi har 15 daqiqada qayta login qiladi. Yechim — ikki token:

- **Access token** — qisqa muddatli (5–15 daqiqa), har bir API so'rovida ishlatiladi.

- **Refresh token** — uzoq muddatli (7–30 kun), faqat `/auth/refresh` ga yuboriladi va yangi access token olish uchun ishlatiladi. U **serverda (bazada) qayd qilinadi** — shuning uchun uni bekor qilish (logout, "barcha qurilmalardan chiqish") mumkin.
- **Rotatsiya** — har refresh'da eski refresh token bekor qilinib, yangisi beriladi. Eski token qayta ishlatilsa — o'g'irlik belgisi, foydalanuvchining barcha sessiyalari yopiladi.

Login xavfsizligi bo'yicha qoidalar

- **Umumiy xato xabari:** "Email yoki parol noto'g'ri" — "Bunday email yo'q" emas (aks holda hujumchi qaysi emaillar ro'yxatdan o'tganini biladi).
- **Rate limiting:** bir IP/akkauntga daqiqada 5–10 urinish (brute-force'ga qarshi).
- **Parol siyosati:** kamida 8–12 belgi; murakkablik qoidalaridan ko'ra uzunlik muhimroq; mashhur sizib chiqqan parollarni rad etish.
- **HTTPS** — majburiy.
- Parolni tiklash tokenlari — bir martalik, qisqa muddatli, bazada xesh ko'rinishida.
- Muhim ilovalar uchun **ikki bosqichli autentifikatsiya (2FA)**: TOTP (Google Authenticator) yoki SMS.
- Tayyor yechimlar ham bor: **OAuth** ("Google bilan kirish"), Auth0, Clerk, Supabase Auth, Better Auth. Lekin ular ortidagi tamoyillarni bilish baribir zarur.

Amaliy misollar

MISOL 1

Ro'yxatdan o'tish va login (bcrypt + JWT + httpOnly cookie)

Terminal

```
npm install bcrypt jsonwebtoken cookie-parser zod
```

migrations/002_users.sql

```
CREATE TABLE users (  
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  name        TEXT NOT NULL,  
  email       TEXT NOT NULL,  
  password_hash TEXT NOT NULL,  
  role        TEXT NOT NULL DEFAULT 'user' CHECK (role IN ('user',  
'editor', 'admin')),  
  token_version INTEGER NOT NULL DEFAULT 0,      -- oshirilsa barcha  
tokenlar bekor bo'ladi  
  created_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
CREATE UNIQUE INDEX users_email_uq ON users (lower(email));
```

src/config.js

```
function required(name) {  
  const v = process.env[name];  
  if (!v) throw new Error(`Muhit o'zgaruvchisi o'rnatilmagan: ${name}`);  
  return v;  
}  
  
export const config = {  
  isProd: process.env.NODE_ENV === 'production',  
  jwtSecret: required('JWT_SECRET'),          // kamida 32 belgili  
tasodifiy satr  
  accessTtl: '15m',  
  clientUrl: process.env.CLIENT_URL ?? 'http://localhost:5173',  
};  
// Kalit yaratish: node -e  
"console.log(require('crypto').randomBytes(48).toString('hex'))"
```

```

import bcrypt from 'bcrypt';
import jwt from 'jsonwebtoken';
import { query } from '../db/pool.js';
import { config } from '../config.js';
import { HttpError } from '../utils/errors.js';

const BCRYPT_COST = 12;
// Foydalanuvchi topilmasa ham bcrypt.compare bajarilishi uchun – javob
vaqti bo'yicha email borligini bilib bo'lmasin
const DUMMY_HASH = await bcrypt.hash('dummy-password', BCRYPT_COST);

const publicUser = ({ id, name, email, role }) => ({ id, name, email, role
});

export function signAccessToken(user) {
  return jwt.sign(
    { sub: String(user.id), role: user.role, tv: user.token_version },
    config.jwtSecret,
    { expiresIn: config.accessTtl, algorithm: 'HS256' }
  );
}

export async function register({ name, email, password }) {
  const passwordHash = await bcrypt.hash(password, BCRYPT_COST);
  try {
    const { rows: [user] } = await query(
      `INSERT INTO users (name, email, password_hash) VALUES ($1,
lower($2), $3)
      RETURNING id, name, email, role, token_version`,
      [name, email, passwordHash]
    );
    return user;
  } catch (err) {
    if (err.code === '23505') throw HttpError.conflict('Bu email bilan
ro\'yxatdan o\'tilgan');
    throw err;
  }
}

export async function login({ email, password }) {
  const { rows: [user] } = await query('SELECT * FROM users WHERE email =
lower($1)', [email]);

```

```
    const ok = await bcrypt.compare(password, user?.password_hash ??  
DUMMY_HASH);  
    if (!user || !ok) throw HttpError.unauthorized('Email yoki parol  
 noto\'g\'ri');  
    return user;  
  }  
  
  export { publicUser };
```



```

import { Router } from 'express';
import { z } from 'zod';
import { validate } from '../middleware/validate.js';
import { requireAuth } from '../middleware/auth.js';
import * as auth from '../services/auth.service.js';
import { config } from '../config.js';
import { rateLimit } from '../middleware/common.js';

const router = Router();

const registerSchema = z.object({
  name: z.string().trim().min(2).max(60),
  email: z.string().trim().email(),
  password: z.string().min(8, 'Parol kamida 8 belgi').max(72), // bcrypt
  // 72 baytdan keyingisini e'tiborsiz qoldiradi
});
const loginSchema = z.object({ email: z.string().trim().email(), password:
z.string().min(1) });

const cookieOptions = {
  httpOnly: true,
  secure: config.isProd,
  sameSite: 'lax',
  maxAge: 15 * 60 * 1000,
};

function sendSession(res, user, status = 200) {
  res.cookie('access_token', auth.signAccessToken(user), cookieOptions);
  res.status(status).json({ user: auth.publicUser(user) });
}

const authLimiter = rateLimit({ windowMs: 15 * 60_000, max: 10 }); // 15
daqiqada 10 urinish

router.post('/register', authLimiter, validate(registerSchema), async
(req, res) => {
  const user = await auth.register(req.validated.body);
  sendSession(res, user, 201);
});

router.post('/login', authLimiter, validate(loginSchema), async (req, res)
=> {

```

```

    const user = await auth.login(req.validated.body);
    sendSession(res, user);
  });

  router.post('/logout', (req, res) => {
    res.clearCookie('access_token', { ...cookieOptions, maxAge: undefined });
  });
  res.status(204).end();
});

router.get('/me', requireAuth, (req, res) => {
  res.json({ user: auth.publicUser(req.user) });
});

export default router;

```

src/app.js (qo'shimcha)

```

import cookieParser from 'cookie-parser';
import authRouter from './routes/auth.routes.js';

app.use(cors({ origin: config.clientUrl, credentials: true })); // cookie
uchun credentials SHART
app.use(cookieParser());
app.use('/api/v1/auth', authRouter);

```

src/middleware/auth.js

```
import jwt from 'jsonwebtoken';
import { query } from '../db/pool.js';
import { config } from '../config.js';
import { HttpError } from '../utils/errors.js';

export async function requireAuth(req, res, next) {
  // Cookie (brauzer) yoki Authorization sarlavhasi (mobil ilova, Postman)
  const header = req.get('authorization');
  const token = req.cookies?.access_token
    ?? (header?.startsWith('Bearer ') ? header.slice(7) : null);

  if (!token) return next(HttpError.unauthorized());

  let payload;
  try {
    payload = jwt.verify(token, config.jwtSecret, { algorithms: ['HS256']
}); // algoritmni aniq ko'rsatamiz
  } catch (err) {
    return next(HttpError.unauthorized(
      err.name === 'TokenExpiredError' ? 'Sessiya muddati tugagan' :
      'Token yaroqsiz'
    ));
  }

  // Foydalanuvchi o'chirilgan yoki "barcha qurilmalardan chiqish"
  qilingan bo'lishi mumkin
  const { rows: [user] } = await query(
    'SELECT id, name, email, role, token_version FROM users WHERE id =
$1',
    [payload.sub]
  );

  if (!user || user.token_version !== payload.tv) return
  next(HttpError.unauthorized('Sessiya bekor qilingan'));

  req.user = user;
  next();
}
```

```
// Ixtiyoriy autentifikatsiya: mehmon ham kira oladi, lekin login qilgan
bo'lsa req.user bo'ladi
export function optionalAuth(req, res, next) {
  if (!req.cookies?.access_token && !req.get('authorization')) return
  next();
  // Token yaroqsiz bo'lsa ham xato bermaymiz – mehmon sifatida davom
  etadi
  return requireAuth(req, res, () => next());
}
```

Endi postlar API'sida egalikni tekshiramiz. **Muhim:** ID'ni so'rov tanasidan emas, `req.user` dan olamiz — aks holda foydalanuvchi boshqa birov nomidan post yaratishi mumkin.

```

import { Router } from 'express';
import { requireAuth } from '../middleware/auth.js';
import { query } from '../db/pool.js';
import { HttpError } from '../utils/errors.js';

const router = Router();

router.post('/', requireAuth, async (req, res) => {
  const { title, body } = req.body; // (validatsiya - 9-
  // darsdagidek)
  const { rows: [post] } = await query(
    'INSERT INTO posts (author_id, title, body) VALUES ($1, $2, $3)
    RETURNING *',
    [req.user.id, title, body] // ✅ author_id -
    // tokendan, mijozdan emas
  );
  res.status(201).json(post);
});

router.patch('/:id', requireAuth, async (req, res) => {
  const { rows: [post] } = await query('SELECT author_id FROM posts WHERE
  id = $1', [req.params.id]);
  if (!post) throw HttpError.notFound('Post topilmadi');

  const isOwner = String(post.author_id) === String(req.user.id);
  if (!isOwner && req.user.role !== 'admin') {
    throw HttpError.forbidden('Faqat muallif yoki admin tahrirlay oladi');
  }

  const { rows: [updated] } = await query(
    'UPDATE posts SET title = COALESCE($2, title), body = COALESCE($3,
    body) WHERE id = $1 RETURNING *',
    [req.params.id, req.body.title, req.body.body]
  );
  res.json(updated);
});

export default router;

```



IDOR — eng keng tarqalgan avtorizatsiya xatosi

`GET /api/orders/1053` — tizimga kirgan foydalanuvchi URL'dagi raqamni o'zgartirib, **boshqa bironing** buyurtmasini ko'rsa — bu "Insecure Direct Object Reference". Har bir resursga murojaatda "bu resurs shu foydalanuvchigami?" degan tekshiruv bo'lishi shart:

```
WHERE id = $1 AND user_id = $2 .
```

MISOL 3

Rollar va ruxsatlar (RBAC)

Har bir rolga emas, har bir **ruxsatga** tekshirish yozish kodni moslashuvchan qiladi: yangi rol qo'shish faqat jadvalni o'zgartirishni talab qiladi.

src/auth/permissions.js

```
export const PERMISSIONS = {
  user: ['post:create', 'post:update:own', 'post:delete:own',
'comment:create'],
  editor: ['post:create', 'post:update:any', 'post:publish',
'comment:create', 'comment:delete:any'],
  admin: ['*'], // hamma narsa
};

export function can(user, permission) {
  const list = PERMISSIONS[user?.role] ?? [];
  return list.includes('*') || list.includes(permission);
}
```

src/middleware/authorize.js

```
import { can } from '../auth/permissions.js';
import { HttpError } from '../utils/errors.js';

export const requirePermission = (...perms) => (req, res, next) => {
  if (!req.user) return next(HttpError.unauthorized());
  if (!perms.some(p => can(req.user, p))) {
    return next(HttpError.forbidden(`Bu amal uchun ruxsat kerak:
    ${perms.join(' yoki ')}`));
  }
  next();
};
```

Foydalanish

```
import { requireAuth } from '../middleware/auth.js';
import { requirePermission } from '../middleware/authorize.js';
import { can } from '../auth/permissions.js';

// Marshrut darajasida
router.post('/:id/publish', requireAuth,
  requirePermission('post:publish'), publishPost);
router.get('/admin/users', requireAuth, requirePermission('*'),
  listUsers);

// Resurs darajasida (egalik bilan)
router.delete('/:id', requireAuth, async (req, res) => {
  const post = await getPost(req.params.id);
  const allowed = can(req.user, 'post:delete:any')
    || (can(req.user, 'post:delete:own') && post.author_id ===
req.user.id);
  if (!allowed) throw HttpError.forbidden();
  await deletePost(post.id);
  res.status(204).end();
});

// Admin foydalanuvchi rolini o'zgartiradi va uning eski tokenlarini bekor
qiladi
router.patch('/admin/users/:id/role', requireAuth, requirePermission('*'),
  async (req, res) => {
    const { rows: [u] } = await query(
      'UPDATE users SET role = $2, token_version = token_version + 1 WHERE
id = $1 RETURNING id, role',
      [req.params.id, req.body.role]
    );
    res.json(u);
  });
```

Frontendda ham xuddi shu `can()` funksiyasidan foydalanib, ruxsat bo'lmagan tugmalarni yashirish mumkin — lekin bu faqat qulaylik uchun. **Haqiqiy tekshiruv doim serverda.**

MISOL 4**Frontend: login formasi, sessiyani tekshirish va 401'ni boshqarish**

httpOnly cookie ishlatilganda frontend tokenni umuman "ko'rmaydi" — brauzer uni o'zi yuboradi. Faqat `credentials: 'include'` kerak.

```

const API = 'http://localhost:3000/api/v1';

export async function api(path, { method = 'GET', body } = {}) {
  const res = await fetch(API + path, {
    method,
    credentials: 'include',          // 🍪 cookie'ni boshqa origin'dagi
API'ga yuborish
    headers: body ? { 'Content-Type': 'application/json' } : {},
    body: body ? JSON.stringify(body) : undefined,
  });

  if (res.status === 401 && path !== '/auth/me' && path !== '/auth/login')
  {
    // Sessiya tugagan – foydalanuvchini login sahifasiga, qaytish manzili
    bilan
    location.href = `/login.html?
next=${encodeURIComponent(location.pathname)}`;
    return new Promise(() => {});    // yo'naltirish paytida kod davom
    etmasin
  }

  const data = res.status === 204 ? null : await res.json();
  if (!res.ok) throw Object.assign(new Error(data?.error?.message ??
'Xatolik'), { status: res.status, data });
  return data;
}

export const auth = {
  me: () => api('/auth/me').then(d => d.user).catch(() => null),
  login: (email, password) => api('/auth/login', { method: 'POST', body: {
email, password } }),
  register: (data) => api('/auth/register', { method: 'POST', body: data
}),
  logout: () => api('/auth/logout', { method: 'POST' }),
};

```

```

<form id="login">
  <h1>Kirish</h1>
  <input name="email" type="email" autocomplete="email"
placeholder="Email" required>
  <input name="password" type="password" autocomplete="current-password"
placeholder="Parol" required>
  <button>Kirish</button>
  <p id="error" role="alert"></p>
</form>

<script type="module">
  import { auth } from './auth.js';

  const form = document.querySelector('#login');
  const error = document.querySelector('#error');

  // Allaqachon kirgan bo'lsa — to'g'ridan-to'g'ri ichkariga
  if (await auth.me()) location.replace('/dashboard.html');

  form.addEventListener('submit', async (e) => {
    e.preventDefault();
    const btn = form.querySelector('button');
    btn.disabled = true;
    btn.textContent = 'Tekshirilmoqda...';
    error.textContent = '';

    try {
      const { email, password } = Object.fromEntries(new FormData(form));
      await auth.login(email, password);
      const next = new URLSearchParams(location.search).get('next');
      // 🛡️ Open redirect'dan himoya: faqat o'z saytimizdagi yo'llarga
      location.href = next?.startsWith('/') && !next.startsWith('//') ?
next : '/dashboard.html';
    } catch (err) {
      error.textContent = err.status === 429
        ? 'Juda ko\'p urinish. 15 daqiqadan so\'ng qayta urinib ko\'ring.'
        : err.message;
      form.elements.password.value = '';
      form.elements.password.focus();
    } finally {
      btn.disabled = false;
      btn.textContent = 'Kirish';
    }
  });

```

```
}  
});  
</script>
```

client/dashboard.html (himoyalangan sahifa)

```
<header>  
  <span id="hello"></span>  
  <button id="logout">Chiqish</button>  
</header>  
<script type="module">  
  import { auth } from './auth.js';  
  
  const user = await auth.me();  
  if (!user) location.replace('/login.html?next=/dashboard.html');  
  else {  
    document.querySelector('#hello').textContent = `Salom, ${user.name}!  
    (${user.role})`;   
    document.querySelector('#logout').onclick = async () => {  
      await auth.logout();  
      location.replace('/login.html');  
    };  
  }  
</script>
```



Cookie va turli domenlar

Frontend `localhost:5173`, API `localhost:3000` bo'lsa ham ular "same-site" (bir xil `localhost`), shuning uchun `SameSite=Lax` cookie ishlaydi. Production'da frontend va API'ni bitta domen ostida (`example.uz` va `api.example.uz`) yoki bitta origin'da (`/api` proksi orqali, 13-dars) joylash eng oson yo'l.

Mashqlar



Mustaqil ish

1. Refresh token mexanizmini qo'shing: `refresh_tokens(id, user_id, token_hash, expires_at, revoked_at)` jadvali, `POST /auth/refresh` (rotatsiya bilan), `POST /auth/logout-all`.
2. "Parolni unutdim": `POST /auth/forgot` bir martalik token yaratib (bazada xeshi bilan, 30 daqiqa muddat) konsolga havola chiqarsin; `POST /auth/reset` uni tekshirib, parolni almashtirsin va `token_version` ni oshirsin.
3. Login urinishlarini `login_attempts` jadvalida qayd qiling; 5 ta muvaffaqiyatsiz urinishdan keyin akkauntni 15 daqiqaga bloklang.
4. jwt.io saytida o'z tokeningizni oching va payload'ni o'zgartirib ko'ring. Server uni nega qabul qilmaydi?
5. `GET /api/orders/:id` marshrutida IDOR zaifligini ataylab yarating, so'ng uni tuzating va farqni test bilan isbotlang.



Xulosa

- AuthN — "kimsan?" (401), AuthZ — "nima qila olasan?" (403).
- Parollar faqat bcrypt/argon2 bilan xeshlangan holda saqlanadi.
- JWT imzolangan, lekin shifrlanmagan; qisqa muddat + refresh token; algoritmni aniq ko'rsating.
- Veb-ilovalarda tokenni `HttpOnly; Secure; SameSite` cookie'da saqlang.
- Foydalanuvchi ID'si doim tokendan; har bir resursda egalikni tekshiring (IDOR).
- Ruqsatlar asosidagi RBAC kodni moslashuvchan qiladi; frontend tekshiruvi faqat UX uchun.

☐ Darsni o'qib bo'ldim